

ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ ΑΣΦΑΛΕΙΑ ΠΛΗΡΟΦΟΡΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΕΓΧΕΙΡΙΔΙΟ ΠΡΟΓΡΑΜΜΑΤΙΣΤΗ 2026

ΓΙΟΥΡΙΙ ΟΣΙΠΙΑΝ Π22125

ΙΩΑΝΝΑ ΑΝΔΡΙΑΝΟΥ Π22010

ΠΕΡΙΕΧΟΜΕΝΑ

1. Μελέτη ασφάλειας ΠΣ
2. Κρυπτογράφηση SSL
3. Authentication / Authorization
 - A) Μηχανισμός Αυθεντικοποίησης
 - B) Έλεγχος πρόσβασης
4. Input Filtering
5. Εύρεση ευπαθειών ασφάλειας (ZAP)
6. Τελική Μελέτη ασφάλειας ΠΣ

1. Μελέτη ασφάλειας ΠΣ

Η εφαρμογή προσφέρει τη δυνατότητα καταχώρησης και διαχείρισης ραντεβού μεταξύ ασθενών και ιατρών, που επιτρέπει τη γρήγορη αναζήτηση ειδικών ιατρών βάσει ειδικότητας και περιοχής, την προβολή της διαθεσιμότητάς τους και την πραγματοποίηση ή ακύρωση ραντεβού από πλευράς του ασθενούς.

Η εφαρμογή διαθέτει διαφορετικές λειτουργίες ανάλογα με το αν ο χρήστης είναι ιατρός ή ασθενής.

Μερικές βασικές λειτουργίες είναι:

- 1. Εγγραφή και Σύνδεση Χρήστη
- 2. Κλείσιμο ραντεβού με γιατρούς
- 3. Οργάνωση μελλοντικών ραντεβού για ασθενείς και γιατρούς
- 4. Ανώνυμες κριτικές για τους γιατρούς από τους ασθενείς

Οι τεχνολογίες πάνω στις οποίες έχει υλοποιηθεί η παραπάνω υπηρεσία είναι οι ακόλουθες:

- * Λειτουργικό Σύστημα: **Debian 12 (Docker 28.5.1)**
- * Εξυπηρετητής Ιστού: **NGINX v1.28**
- * Εξυπηρετητής εφαρμογής: **JDK 21 Jetty v11.0.25 (embedded via Javalin 6.6.0)**
- * Εξυπηρετητής βάσης δεδομένων: **SQLite 3.49.1.0, ActiveJDBC 3.5-j11 (ORM)**

Asset Model

Υπολογιστικό Σύστημα: Web Server

HW	Server (μοντέλο, χαρακτηριστικά) Τοποθεσία (κτήριο, δωμάτιο)	Docker Greece
SW	Λειτουργικό Σύστημα (πυρήνας, έκδοση) Λογισμικό Εφαρμογών Άλλο Λογισμικό	Debian 12 x86-64 Docker Container 28.5.1 NGINX
Network	Περιοχή Δικτύου (network zone) Σημείο Σύνδεσης (Gateway)	localhost 9191
Data	Δεδομένα Διαμόρφωσης (Configuration data) Δεδομένα Λειτουργίας υπηρεσιών (Operation data) Άλλα δεδομένα	NGINX-config, Docker-config, versions-info, connections-info certificates private-key web-structure images

Υπολογιστικό Σύστημα: Backend Server (Application)

HW	Server (μοντέλο, χαρακτηριστικά) Τοποθεσία (κτήριο, δωμάτιο)	Docker Greece
SW	Λειτουργικό Σύστημα (πυρήνας, έκδοση) Λογισμικό Εφαρμογών Άλλο Λογισμικό	Debian 12 x86-64 Docker Container 28.5.1 JDK-21 Jetty v11.0.25 Javalin 6.6.0 (web framework), Active JDBC 3.5-j11 (ORM)
Network	Περιοχή Δικτύου (network zone) Σημείο Σύνδεσης (Gateway)	localhost 7070
Data	Δεδομένα Διαμόρφωσης (Configuration data) Δεδομένα Λειτουργίας υπηρεσιών (Operation data) Άλλα δεδομένα	Git Docker-config maven-config Application-code(bru swl java json) backend-structure Documentations, diagrams

Υπολογιστικό Σύστημα: Backend Server (Database)

HW	Server (μοντέλο, χαρακτηριστικά) Τοποθεσία (κτήριο, δωμάτιο)	Docker Greece
SW	Λειτουργικό Σύστημα (πυρήνας, έκδοση) Λογισμικό Εφαρμογών Άλλο Λογισμικό	Debian 12 x86-64 Docker Container 28.5.1 NGINX SQLite 3.49.1.0
Network	Περιοχή Δικτύου (network zone) Σημείο Σύνδεσης (Gateway)	Localhost 7070
Data	Δεδομένα Διαμόρφωσης (Configuration data) Δεδομένα Λειτουργίας υπηρεσιών (Operation data) Άλλα δεδομένα	SQLite calls Database -

Υπηρεσία - Υπολογιστικά Συστήματα

Και οι τέσσερις υπηρεσίες που παρέχονται από το συγκεκριμένο σύστημα χρησιμοποιούν τον web server, τον application server και τη βάση δεδομένων.

	Web Server	Application Server	Database
Εγγραφή και Σύνδεση Χρήστη	✓	✓	✓
Κλείσιμο ραντεβού με γιατρούς	✓	✓	✓
Οργάνωση μελλοντικών ραντεβού για ασθενείς και γιατρούς	✓	✓	✓
Ανώνυμες κριτικές για τους γιατρούς από τους ασθενείς	✓	✓	✓

- Αρχικά, για την εγγραφή και τη σύνδεση χρήστη πρέπει να χρησιμοποιηθούν τα login credentials του χρήστη που είναι αποθηκευμένα στη βάση δεδομένων ή να αποθηκευτούν νέα. Για αυτό, από τη σελίδα login ή sign up στο web, τα στοιχεία στέλνονται στο backend που ελέγχει τη βάση δεδομένων (στη σύνδεση) ή αποθηκεύει τα στοιχεία του νέου χρήστη (στην εγγραφή).
- Για το κλείσιμο ραντεβού χρειάζεται να γίνει retrieve από το backend τα στοιχεία των γιατρών από τη βάση καθώς και οι διαθέσιμες ώρες τους.
- Για την οργάνωση των μελλοντικών ραντεβού σε calendar ζητάμε από τη βάση τα ραντεβού του συγκεκριμένου χρήστη.
- Για να αφήσει κριτική ο χρήστης θα πρέπει να την αποθηκεύσουμε στη βάση και για να τη δει ο γιατρός θα την κάνουμε retrieve από τη βάση.

Impact Assessment

Υπηρεσία: Εγγραφή και Σύνδεση Χρήστη

Συνέπειες για:	Τύπος Συνέπειας	Βαθμός Συνέπειας	Σύντομη Αιτιολόγηση
Μη διαθεσιμότητα (Unavailability)	Παραμπόδιση λειτουργιών Άμεσες οικονομικές απώλειες Δυσφήμιση	Υψηλή	Η αδυναμία σύνδεσης των χρηστών μπορεί να οδηγήσει σε απώλεια εσόδων λόγω αδυναμίας κλεισίματος ραντεβού καθώς και σε δυσφήμιση της εταιρίας.

Αποκάλυψη Δεδομένων (Disclosure)	Δυσφήμιση Νομικές κυρώσεις	Υψηλή	Η αποκάλυψη login credentials ή ιατρικών δεδομένων μπορεί να οδηγήσει σε σοβαρές νομικές συνέπειες.
Τροποποίηση Δεδομένων (Modification)	Παραμπόδιση λειτουργιών Δυσφήμιση	Μέτρια	Η τροποποίηση των στοιχείων σύνδεσης των χρηστών μπορεί να προκαλέσει προβλήματα στη χρήση της εφαρμογής και απώλεια εμπιστοσύνης.

Υπηρεσία: Κλείσιμο ραντεβού με γιατρούς

Συνέπειες για:	Τύπος Συνέπειας	Βαθμός Συνέπειας	Σύντομη Αιτιολόγηση
Μη διαθεσιμότητα (Unavailability)	Άμεσες οικονομικές απώλειες Παραμπόδιση λειτουργιών Δυσφήμιση	Υψηλή	Ως βασική λειτουργία της εφαρμογής, η μη διαθεσιμότητά της θα οδηγήσει σε απώλεια εσόδων λόγω της αδυναμίας κλεισίματος ραντεβού.
Αποκάλυψη Δεδομένων (Disclosure)	Δυσφήμιση Νομικές κυρώσεις	Μέτρια	Λόγω ιατρικού απορρήτου, η αποκάλυψη στοιχείων των ραντεβού μπορεί να οδηγήσει σε νομικά προβλήματα.
Τροποποίηση Δεδομένων (Modification)	Δυσφήμιση Παραμπόδιση λειτουργιών	Μέτρια	Η τροποποίηση των στοιχείων ενός ραντεβού μπορεί να

			προκαλέσει ασυνεννοησία μεταξύ γιατρού και ασθενούς.
--	--	--	--

Υπηρεσία: Οργάνωση μελλοντικών ραντεβού

Συνέπειες για:	Τύπος Συνέπειας	Βαθμός Συνέπειας	Σύντομη Αιτιολόγηση
Μη διαθεσιμότητα (Unavailability)	Δυσφήμιση Παρεμπόδιση λειτουργιών	Μέτρια	Οι χρήστες δεν θα μπορούν να οργανώσουν τα μελλοντικά τους ραντεβού, κάτι που μπορεί να οδηγήσει σε ασυνεννοησία μεταξύ γιατρών και ασθενών.
Αποκάλυψη Δεδομένων (Disclosure)	Δυσφήμιση Νομικές κυρώσεις	Υψηλή	Η αποκάλυψη μελλοντικών ραντεβού μπορεί να οδηγήσει σε νομικές κυρώσεις λόγω ιατρικού απορρήτου.
Τροποποίηση Δεδομένων (Modification)	Δυσφήμιση Παρεμπόδιση λειτουργιών	Μέτρια	Η τροποποίηση των στοιχείων ενός ραντεβού μπορεί να προκαλέσει ασυνεννοησία μεταξύ γιατρών και ασθενών.

Υπηρεσία: Ανώνυμες κριτικές για τους γιατρούς

Συνέπειες για:	Τύπος Συνέπειας	Βαθμός Συνέπειας	Σύντομη Αιτιολόγηση
Μη διαθεσιμότητα (Unavailability)	Δυσφήμιση	Χαμηλή	Μπορεί να οδηγήσει σε λιγότερα ραντεβού για τους γιατρούς και σε

			δυσφήμιση για την εταιρία.
Αποκάλυψη Δεδομένων (Disclosure)	Δυσφήμιση	Χαμηλή	Η αποκάλυψη της ταυτότητας των σχολιαστών που υποτίθεται ότι είναι ανώνυμοι μπορεί να προκαλέσει δυσφήμιση.
Τροποποίηση Δεδομένων (Modification)	Δυσφήμιση Νομικές κυρώσεις	Μέτρια	Η τροποποίηση κριτικών μπορεί να οδηγήσει σε εσφαλμένη δυσφήμιση κάποιου γιατρού και πιθανές νομικές συνέπειες.

Threat Assessment

Threat:	Web Server:	Application Server	Database
Unauthorized Access	Medium	Medium	Low
Ransomware	Low	Medium	Medium
Web Defacement	High	Low	Low
Code Injection	High	Medium	Low
Denial of Service	High	Medium	Low

Σχόλια: Ο πίνακας έχει γίνει με βάση την πιθανότητα που υπάρχει κάποια απειλή να εκδηλωθεί σε κάποιον από τους server. Γενικά το database βρίσκεται πίσω από το backend οπότε είναι πιο δύσκολο να γίνει επίθεση προς αυτόν ενώ ο web server και το backend είναι πολύ πιο ευάλωτοι. Επιπλέον το web defacement και το code injection γίνονται συνήθως μέσω του frontend για αυτό και έχουν μεγαλύτερη πιθανότητα να εκδηλωθεί εκεί. Τέλος τα περισσότερα DoS attacks γίνονται με επίθεση στο frontend.

Vulnerability Assessment

Με την βάση αδυναμιών ασφαλείας του NIST και υπολογίζοντας το CVSS 3 score κάθε vulnerability έχουμε τον παρακάτω πίνακα:

Λογισμικό	Αδυναμία	CVSS 3.1 Score
Debian 12 sudo 1.9.13p3	CVE-2025-32463	7.8 (High)
Docker Compose 2.40.0	CVE-2025-62725	8.9 (High)
JDK 21	CVE-2024-20952	7.4 (High)
Eclipse Jetty 11.0.5	CVE-2025-5115	7.7 (High)
Eclipse Jetty 11.0.5	CVE-2021-34429	5.3 (Medium)

Σημείωση: Για NGINX 1.28, SQLite 3.49.1 και ActiveJDBC 3.5-j11 δεν βρέθηκαν αδυναμίες.

Risk Assessment

A) Αποτίμηση Απειλών

Παρατηρούμε ότι παρόλο που ο αριθμός των απειλών που βρήκαμε στο vulnerability assessment είναι χαμηλός, στο σύστημα μας υπάρχουν κάποια σοβαρά vulnerabilities (severity: High), τα οποία αυξάνουν την αποτίμηση των απειλών. Για το cvss για κάθε απειλή χρησιμοποιήσαμε τον μέσο όρο των απειλών καθώς στην πιθανότητα απειλής βάλαμε το υψηλότερο που υπήρχε σε κάθε απειλή στο threat assessment. Ο πίνακας για την αποτίμηση επιπέδου απειλής είναι ο παρακάτω:

Επίπεδο Αδυναμίας (βάσει CVSS score): -->	None	Low	Medium	High	Critical
Πιθανότητα Απειλής : v					
0 (Low)	0	0	0	1	1
1 (Medium)	0	1	2	2	3
2 (High)	0	1	2	3	4

Οπότε με βάση τον πίνακα έχουμε τις παρακάτω αποτιμήσεις:

Απειλή	Αδυναμίες	Μέσος όρος CVSS Score	Πιθανότητα Απειλής	Αποτίμηση Επιπέδου Απειλής
Unauthorized Access	CVE-2025-32463 CVE-2024-20952 CVE-2021-34429	7.5 (High)	Medium	2
Ransomware	CVE-2025-62725	8.9 (High)	Medium	2
Web Defacement	CVE-2021-34429	5.3 (Medium)	High	2
Code Injection	CVE-2025-62725	8.9 (High)	High	3
Denial of Service	CVE-2025-5115	7.7 (High)	High	3

Παρατηρούμε ότι οι περισσότερες αδυναμίες του συστήματός μας είναι στην απειλή Unauthorized Access.

Υπό άλλες συνθήκες, ενώ η συγκεκριμένη απειλή είναι πιο δύσκολη να εκδηλωθεί, παρατηρούμε ότι αυξάνεται η πιθανότητα λόγω του πλήθους των αδυναμιών. Στην συγκεκριμένη περίπτωση όμως, μιας και η πιθανότητα απειλής είχε υπολογιστεί ανεξάρτητα των vulnerabilities, παραμένει Medium.

B) Αποτίμηση Κινδύνου

Στην αποτίμηση κινδύνου είναι σημαντικό να αναφερθεί ότι το συγκεκριμένο σύστημα χρησιμοποιεί σημαντικά προσωπικά δεδομένα και για αυτό είναι πολύ σημαντική η ασφαλής και σωστή λειτουργία του συστήματος. Για αυτό το λόγο, το επίπεδο συνέπειας παραμένει γενικά υψηλό.

Όπως αναφέρθηκε και νωρίτερα στο impact assessment, λόγω ιατρικού απορρήτου που μπορεί να οδηγήσει σε νομικές κυρώσεις η συνέπειες μπορούν να είναι αρκετά μεγάλες.

Οπότε χρησιμοποιώντας τον παρακάτω πίνακα:

Αποτίμηση Απειλής: -->	0	1	2	3	4
Αποτίμηση Συνέπειας: v					
1 (Low)	0	1	2	3	4

2 (Medium)	0	2	4	6	8
3 (High)	0	3	6	9	12

Έχουμε τον παρακάτω πίνακα αποτίμησης κινδύνου:

Απειλή	Επίπεδο Συνέπειας	Αποτίμηση Απειλής	Αποτίμηση επιπέδου κινδύνου
Unauthorized Access	High (Πληροφορίες χρηστών, ιατρικό απόρρητο)	2	6
Ransomware	High (Καταστροφή δεδομένων, μη λειτουργικότητα συστήματος)	2	6
Web Defacement	Medium (Δυσφήμιση εταιρίας)	2	3
Code Injection	High (Καταστροφή δεδομένων, unauthorized access)	3	6
Denial of Service	High (Μη λειτουργικότητα του συστήματος)	3	6

Risk Threshold

Με κατώφλι κινδύνου να είναι 2 παρατηρούμε ότι όλες οι απειλές είναι πάνω από το κατώφλι κινδύνου. Γενικά παρατηρούμε ότι το σύστημα έχει αποτίμηση κινδύνου επίπεδο ~6 . Αυτό θεωρείται υψηλό αλλά είναι λογικό λόγω των ευαίσθητων στοιχείων και λόγω των vulnerabilities που βρήκαμε. Αυτό σημαίνει ότι για να μειωθεί ο κίνδυνος θα πρέπει μελλοντικά να μειώσουμε, όσο το δυνατόν, τα vulnerabilities που υπάρχουν ώστε η αποτίμηση της απειλής να είναι όσο χαμηλά γίνεται.

2. Κρυπτογράφηση SSL (3η άσκηση 4-5 βημ)

Εφόσον έχουν γίνει τα βήματα 1-2-3 και δεν ζητείται αναφορά σε αυτά, θα παραληφθούν σε αυτό το έγγραφο. Σε περίπτωση ανάγκης, στο thales.cs.unipi.gr υπάρχει ανεβασμένο αρχείο με όνομα “asfaleia_ergasia_3.zip” στην εργασία “3η Άσκηση - Δημιουργία Αρχών Πιστοποίησης και διαχείριση πιστοποιητικών”

4)Εισαγωγή πιστοποιητικού στον server

Από το ερωτήματα 1 & 2 υπάρχουν οι αρχές πιστοποιήσεις (ca – certificatoin authorities) και τα πιστοποιητικά τους. Από το 3 ερώτημα έχουμε το πιστοποιητικό για τον Web Server. Τώρα για την πρόσθεση του πιστοποιητικού στον Web Server, θα χρειαστεί να τροποποιηθεί το configuration file του nginx:

```
server {
    listen 9191 ssl;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    ssl_certificate /etc/nginx/ssl/ca-chain.cert.pem;
    ssl_certificate_key /etc/nginx/ssl/server.key.pem;
}
```

Όπως φαίνεται και από την εικόνα έχει προσθέσει “server.key.pem” το οποίο είναι private key του server και “ca-chain.cert.pem” η οποία είναι αλυσίδα που αποτελείτε από

- Example Intermediate CA
- localhost (Web Server certification)

Σε περίπτωση που κάποιος χρήστης θελήσει να μπει στην σελίδα του Web Server και δεν έχει εμπιστοσύνη σε κάποιο από τα 3 certifications που έχουν δημιουργηθεί (Example Root CA - Example Intermediate CA - localhost). Θα εμφανίζονται μηνύματα προειδοποιήσεις μη ασφαλής ιστοσελίδα.

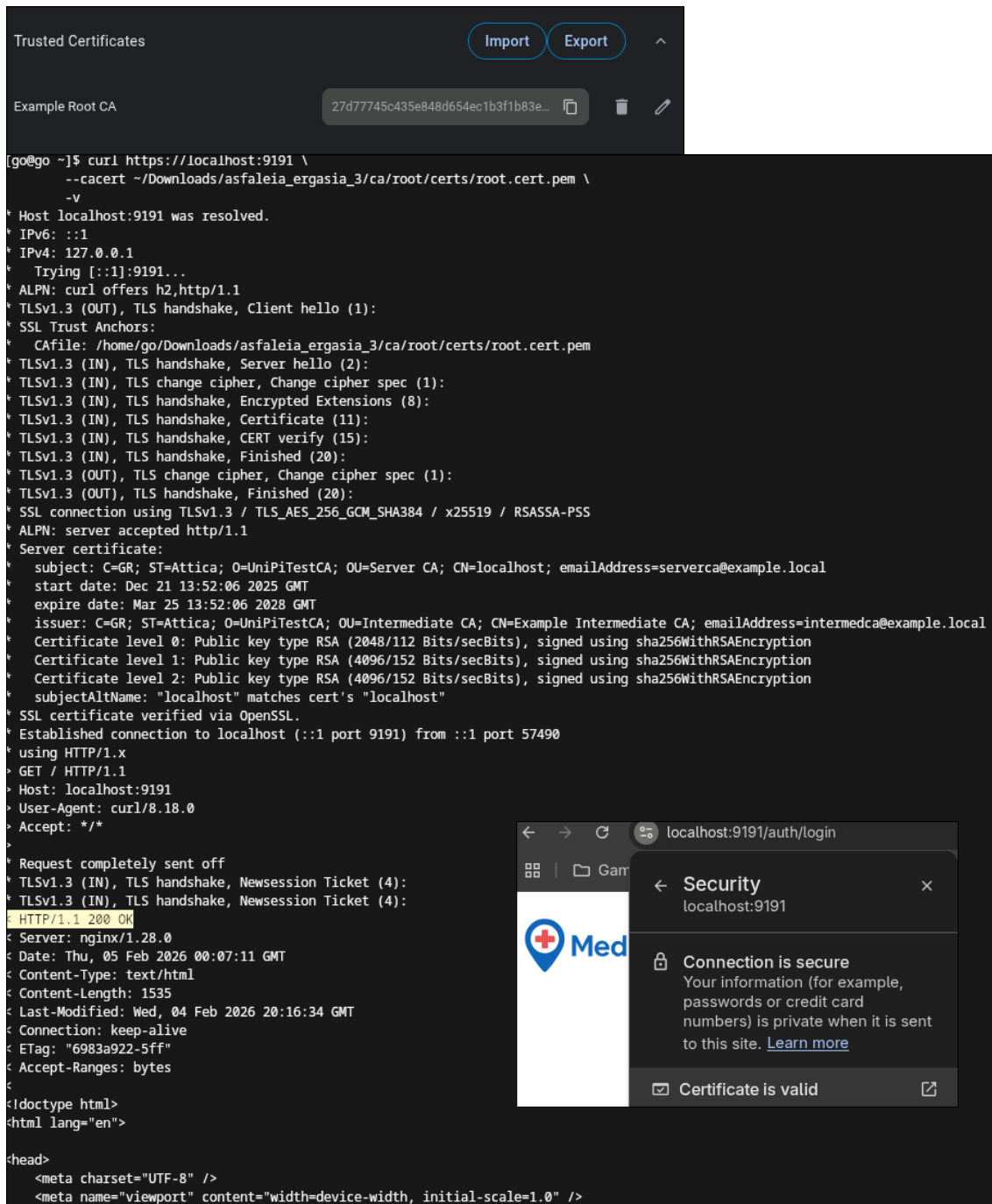
Θα χρειαστεί να αναφερθεί ότι επειδή εμφανίστηκε πρόβλημα:

Error: "This server could not prove that it is localhost; its security certificate does not specify Subject Alternative Names. This may be caused by a misconfiguration or an attacker intercepting your connection."

Προστέθηκε η γραμμή “subjectAltName = DNS:localhost, IP:127.0.0.” στο αρχείο “intermediate_openssl.cnf” με στόχο

- Να υπάρχει πλαίσιο “subjectAltName” μη κενό
- Να γίνεται σωστή συσχέτιση ονομάτων με τον Web Server

Στιγμιότυπα που παρουσιάζουν την λειτουργικότητα:



5) Διαμόρφωση του server για διπλή αυθεντικοποίηση

Για να λειτουργήσει η διπλή αυθεντικοποίηση θα χρειαστεί να προστεθούν ρυθμίσεις στο nginx. Δηλαδή αυτό που θα προστεθεί είναι τα certification θα εμπιστεύεται ο Web Server

```
ssl_client_certificate /etc/nginx/ssl/ca-chain-verification.cert.pem;  
ssl_verify_client on;  
ssl_verify_depth 2;
```

Το αρχείο “ca-chain-verification.cert.pem” είναι η αλυσίδα από

- Example Root CA

- Certificate για τον client με βάση του “intermediate_openssl.cnf”

```
(tarmac@durren)-[~/ca/client]
$ openssl req -new -key "$SCL_DIR/client.key.pem" -out "$SCL_DIR/client.csr.pem" -config "$INT_DIR/intermediate_openssl.cnf"
You are about to be asked to enter information that will be incorporated into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter) [GR]:
State or Province Name [Attica]:
Locality Name [Piraeus]:
Organization Name [UniPiTestCA]:Tarmac
Organizational Unit Name [Intermediate CA]:Tarmac
Common Name [Example Intermediate CA]:Tarmac
Email Address [intermedca@example.local]:tarmac@example.local
```

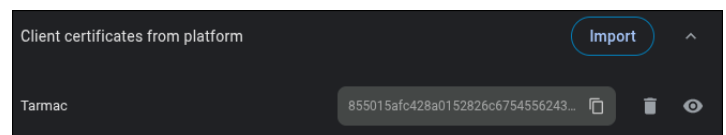
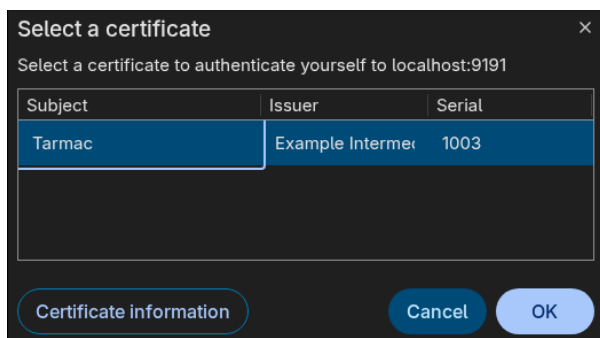
- Υπογραφή certificate του client από τον intermediate με την ρύθμιση “-extension usr_cert”, ώστε ο intermediate να ακολουθήσει τους κανόνες από το παιδί usr_cert από το “intermediate_openssl.cnf”

```
(tarmac@durren)-[~/ca/client]
$ openssl ca -config "$INT_DIR/intermediate_openssl.cnf" -extensions usr_cert -days 825 -notext -md sha256 -in "$SCL_DIR/client.csr.pem" -out "$SCL_DIR/client.cert.pem"
Using configuration from /home/tarmac/ca/intermediate/intermediate_openssl.cnf
Enter pass phrase for /home/tarmac/ca/intermediate/private/intermediate.key.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
  Serial Number: 4099 (0x1003)
  Validity
    Not Before: Dec 21 14:10:36 2025 GMT
    Not After : Mar 25 14:10:36 2028 GMT
  Subject:
    countryName           = GR
    stateOrProvinceName   = Attica
    organizationName      = Tarmac
    organizationalUnitName = Tarmac
    commonName            = Tarmac
    emailAddress          = tarmac@example.local
```

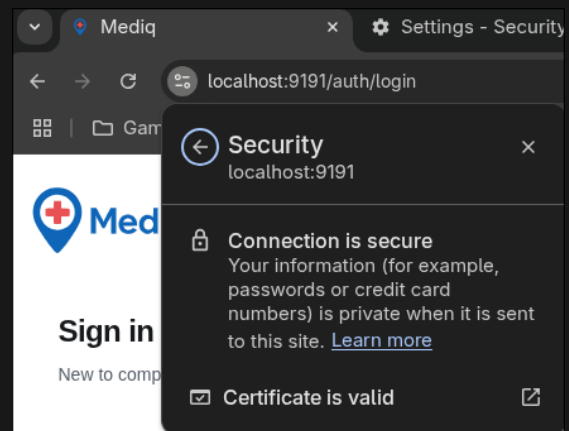
- Bonus βήμα: Έτσι ο browser να μπορέσει να στείλει το certificate του client, θα χρειαστεί το certificate και το private key του client. Έτσι δημιουργείται αρχείο τύπου PKCS#12 για τον web browser

```
[go@go client]$ openssl pkcs12 -export -inkey client.key.pem -in client.cert.pem -out client.p12
Enter Export Password:
Verifying - Enter Export Password:
```

Στιγμιότυπα που δείχνουν την λειτουργικότητα:



```
[go@go ~]$ curl https://localhost:9191/ \
--cert ~/Downloads/asfaleia_ergasia_3/ca/client/client.cert.pem \
--key ~/Downloads/asfaleia_ergasia_3/ca/client/client.key.pem \
--cacert ~/Downloads/asfaleia_ergasia_3/ca/root/certs/root.cert.pem \
-v
* Host localhost:9191 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:9191...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust Anchors:
* CAfile: /home/go/Downloads/asfaleia_ergasia_3/ca/root/certs/root.cert.pem
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Request CERT (13):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Certificate (11):
* TLSv1.3 (OUT), TLS handshake, CERT verify (15):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
* subject: C=GR; ST=Attica; O=UniPiTestCA; OU=Server CA; CN=localhost; emailAddress=serverca@example.local
* start date: Dec 21 13:52:06 2025 GMT
* expire date: Mar 25 13:52:06 2028 GMT
* issuer: C=GR; ST=Attica; O=UniPiTestCA; OU=Intermediate CA; CN=Example Intermediate CA; emailAddress=intermedca@example.local
* Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
* Certificate level 1: Public key type RSA (4096/152 Bits/secBits), signed using sha256WithRSAEncryption
* Certificate level 2: Public key type RSA (4096/152 Bits/secBits), signed using sha256WithRSAEncryption
* subjectAltName: "localhost" matches cert's "localhost"
* SSL certificate verified via OpenSSL.
* Established connection to localhost (::1 port 9191) from ::1 port 38574
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:9191
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/1.1 200 OK
< Server: nginx/1.28.0
< Date: Thu, 05 Feb 2026 01:25:52 GMT
< Content-Type: text/html
< Content-Length: 1535
< Last-Modified: Wed, 04 Feb 2026 20:16:34 GMT
< Connection: keep-alive
< ETag: "6983a922-5ff"
< Accept-Ranges: bytes
<
<!doctype html>
<html lang="en">
```



3. Authentication / Authorization

Σημείωση: Ό,τι δεν έχει σχέση με Authentication και Authorization σε αυτό το κεφάλαιο δεν θα αναφερθεί

A) Μηχανισμός Αυθεντικοποίησης

Οι μηχανισμός αυθεντικοποίησης που υλοποιείτε στην Εφαρμογή είναι username - password πάνω στις τεχνολογίες

- Backend:

- Javalin (Backend Web API)
- JDBC (ORM)
- Frontend:
 - Axios (https requests)
 - Mobx (session storage)

Backend:

Στο Backend υπάρχουν οι κλάσεις controllers που αναλαμβάνουν όλα τα request που θα στέλνει ο χρήστης. Στην περίπτωσή μας, μας ενδιαφέρει ο controller “UserController.java”, ο οποίος αναλαμβάνει τα authentication requests

Έχει τις εξής μεθόδους για την Αυθεντικοποίηση:

- **init(Javalin app);** Ορίζει ποιες μέθοδοι θα ανταποκρίνονται σε ποια requests
- **registerUser(Context context);** Εγγραφή του χρήστη
- **loginUser(Context context); - getLogin(Context context);** Σύνδεση χρήστη και έλεγχο σύνδεσης
- **logoutUser(Context context);** Αποσύνδεση χρήστη
- **updateUser(Context context);** Ανανέωση δεδομένων χρήστη
- **getUser(Context context);** Αποστολή δεδομένων του user

Βοηθητικές μέθοδοι:

- **getUserData(User user);** Επιστρέφει τον user σε μορφή “Has<String, Object>”. Χρησιμοποιείτε από getLogin και LoginUser
- **addIfNotNull(Map<String, Object> map, String key, Object value);** Ανάγνωση και εγγραφή δεδομένων χωρίς κίνδυνο Null τιμών

Τώρα κάθε μέθοδο του “UserController.java” που χρειάζεται επικοινωνία με την βάση, καλεί μεθόδους από τα services και συγκεκριμένα από “UserService.java”.

Τα services αναλαμβάνουν την επικοινωνία με τα models. Με αυτόν τον τρόπο ξεφορτώνουμε τους controllers από λειτουργικότητα

Ο UserService τα models που χρησιμοποιεί είναι:

- Doctor
- Patient
- User

Ο UserService έχει τις εξής μεθόδους για αυθεντικοποίηση:

- **getUser(Registerbody body);** Επιστροφή του χρήστη που ζητείτε
- **registerUser(RegisterBody body);** Έλεγχος ποιος χρήστης εγγράφεται και η εγγραφή του χρήστη στην βάση

- **loginUser(String username, String password);** Έλεγχος ύπαρξης του χρήστη και επιστροφή του

Κάθε model έχει ως στόχο την εύκολη επικοινωνία με την βάση. Τα models επικοινωνούν με το ORM (JDBC) ,το οποίο επικοινωνεί με την βάση. Κάθε model έχει constructor που φτιάχνει μία εγγραφή στον πίνακα και τα αντίστοιχα getters με setters που χρειάζονται τα services.

Frontend:

Για μία πιο απλή HTTPS επικοινωνία με το Backend χρησιμοποιείτε η βιβλιοθήκη “Axios”. Με αυτή ορίζονται δύο αντικείμενα στο αρχείο “src/api/Index.ts”.

- **client:** Παίρνει πάνω του την διαχείριση των 401 requests
- **user:** Συναρτήσεις που παίρνουν πάνω το όλα τα HTTPS requests . Δηλαδή τα authenticatoion requests που προσφέρει είναι:
 - **Post**
 - **register:** Εγγραφή χρήστη
 - **login:** Σύνδεση χρήστη
 - **Get**
 - **getLogin:** Έλεγχο σύνδεσης χρήστη
 - **logout:** Αποσύνδεση χρήστη

Επίσης υπάρχει “src/stores/UserStorage.tsx” το οποίο:

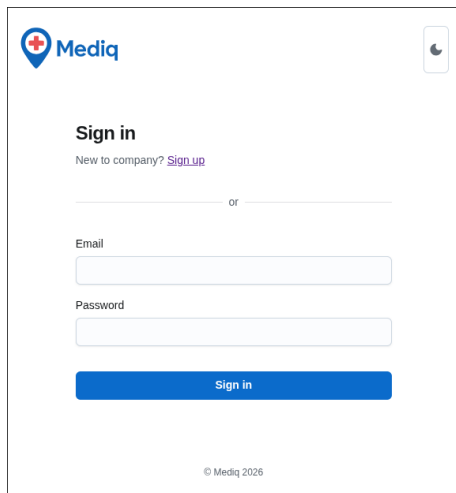
- Αποθηκεύει τον αυθεντικοποιημένο χρήστη στο session μέσα της βιβλιοθήκης “Mobox”
- Στέλνει request καλώντας τον user από το “src/api/Index.ts”

Οι φόρμες εγγραφής και σύνδεσης βρίσκονται στον φάκελο “src/components/”.

- **signup:** Στέλνει request και μετά ανακατευθύνει τον χρήστη στο login
- **signin:** στέλνει request και μετά όταν πάρει απάντηση, ανακατευθύνει στην αντίστοιχη σελίδα
-

Στιγμιότυπα λειτουργικότητας:

- Οι φόρμες



Sign in
New to company? [Sign up](#)

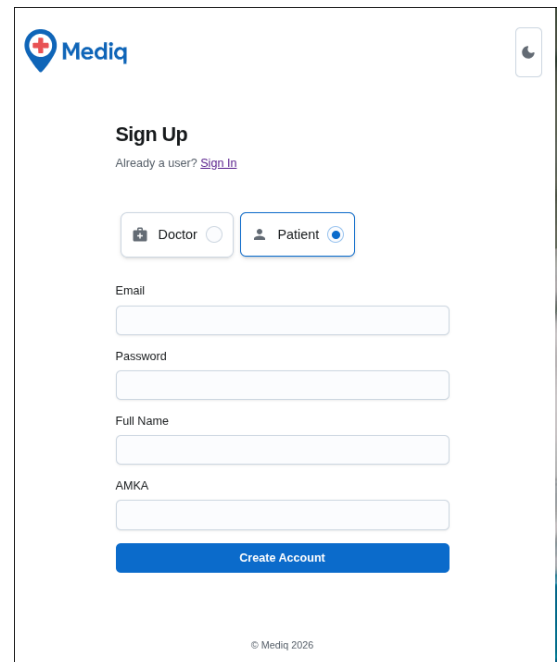
_____ or _____

Email

Password

Sign in

© Mediq 2026



Sign Up
Already a user? [Sign In](#)

☐ Doctor ☒ Patient

Email

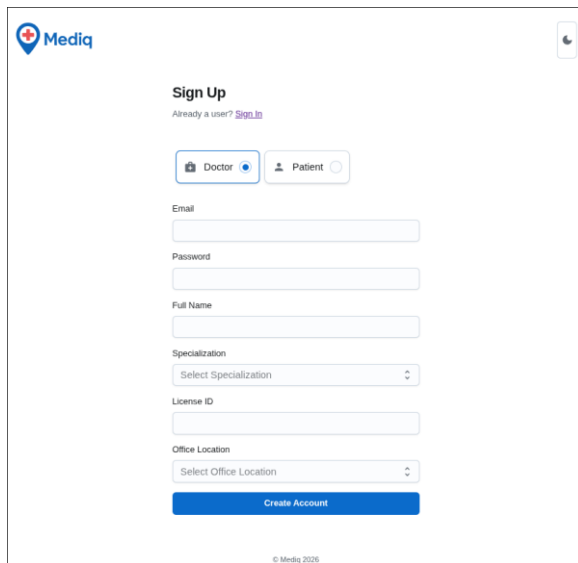
Password

Full Name

AMKA

Create Account

© Mediq 2026



Sign Up
Already a user? [Sign In](#)

☒ Doctor ☐ Patient

Email

Password

Full Name

Specialization
Select Specialization

License ID

Office Location
Select Office Location

Create Account

© Mediq 2026

- Η βάση πριν τις αλλαγές (βλέπουμε κάποια columns και όλα τα rows που υπάρχουν στην βάση)

id	email	password	fullName	userType
9	dentist@mail.com	a	James Sharp-Teeth	DOCTOR
10	impatient@mail.com	a	Klara Dall	PATIENT
11	eyeballing@mail.com	a	Evan Dearryl	DOCTOR
12	sick@mail.com	a	Josh Sick	PATIENT
13	hair@mail.com	a	Co Ming Soon	DOCTOR
14	psychiatry@mail.com	a	Jason	DOCTOR
15	bestdoctor@mail.com	a	Joshua	DOCTOR
16	mimi@gmail.com	mimi	mimi	PATIENT
17	mimi2@gmail.com	mimi2	mimi2	PATIENT

- Εγγραφή και Σύνδεση (Υπάρχει μήνυμα σε περίπτωση σφάλματος)

Sign Up

Already a user? [Sign In](#)

☒ Doctor
 ☒ Patient

Email

test@test.com

Password

•

Full Name

Mr Test

AMKA

4231877289

Create Account



☒ Sign in
 ☐ Sign up

New to company? [Sign up](#)

or

Email

test@test.com

Password

•

Sign in

© Mediq 2022

Cannot register user: User with this email already exists

Cannot login: Invalid email or password



- Home
- My Appointments
- History
- My Diagnoses
- Profile


 Mr Test
 test@test.com




Home

Search for doctors in your area

Specialty: All Specialities
 Location: All Locations


 DENTIST
 James Sharp-Teeth
 Thessaloniki
 3.5


 OPHTHALMOLOGIST
 Evan Dearryl
 Athens
 4.7


 DERMATOLOGIST
 Co Ming Soon
 Patras
 1.0

- Αποσύνδεση


 Mr Test
 test@test.com




- Βάση μετά την εγγραφή. Φαίνεται η 10 εγγραφή που προστέθηκε

	id	email	password	fullName	userType
	Filter...	Filter...	Filter...	Filter...	Filter...
1	9	dentist@mail.com	a	James Sharp-Teeth	DOCTOR
2	10	impatient@mail.com	a	Klara Dall	PATIENT
3	11	eyeballing@mail.com	a	Evan Dearryl	DOCTOR
4	12	sick@mail.com	a	Josh Sick	PATIENT
5	13	hair@mail.com	a	Co Ming Soon	DOCTOR
6	14	psychiatry@mail.com	a	Jason	DOCTOR
7	15	bestdoctor@mail.com	a	Joshua	DOCTOR
8	16	mimi@gmail.com	mimi	mimi	PATIENT
9	17	mimi2@gmail.com	mimi2	mimi2	PATIENT
10	18	test@test.com	a	Mr Test	PATIENT

B) Έλεγχος πρόσβασης

Για να υπάρχει έλεγχος πρόσβασης, πρέπει πρώτα να ορισθούν οι ρόλοι και τι πρόσβαση έχουν.

- **Guest:** Επισκέπτης μπορεί μόνο να Γραφτεί είτε να Συνδεθεί
- **Patient:** Ασθενής έχει την δυνατότητα να
 - Κλείσει ραντεβού με Γιατρό
 - Δει την ιστορία Υγείας του
 - Δει το ιστορικό του με γιατρούς
 - Αφήσει σχόλιο σε κάποιο γιατρό
- **Doctor:** Γιατρός έχει την δυνατότητα να
 - Δει τα ραντεβού που έχει με τους Ασθενείς
 - Ορίσει ποιες μέρες και ώρες είναι ελεύθερος για ραντεβού
 - Δει το ιστορικό του με Ασθενείς
 - Δει τα σχόλια που του έχουν αφήσει οι Ασθενείς

Ο στόχος είναι να μην μπορεί να κάνει κάποιος ρόλος

- Παραπάνω από αυτά που του προσφέρετε
- Λιγότερα από αυτά που του υπόσχονται

Οι μηχανισμοί για τον έλεγχο πρόσβασης είναι session-based και role-based. Είναι πάνω στις ίδιες τεχνολογίες όπως και η αυθεντικοποίηση

- Backend:
 - Javalin (Backend Web API)
 - JDBC (ORM)
- Frontend:
 - Axios (https requests)
 - Mobx (session storage)

Backend:

Για να πραγματοποιηθεί χωρισμός ρόλων. Έχει προστεθεί η κλάση “AuthUtils.java”. Αυτή η κλάση αναλαμβάνει όλους τους ελέγχους πρόσβασης

Η κλάση AuthUtils αποτελείται από της παρούσες μεθόδους:

- **validateUserAndGetId(Context context, UserTypeEnum requiredUserType);** Ελέγχει το session του χρήστη και ελέγχει τον τύπο του χρήστη. Μόνο σε περίπτωση που ο τύπος του χρήστη είναι ίδιος με τον requiredUserType, θα επιστραφεί το ID του χρήστη
- **validateDoctorAndGetId(Context context);** Ελέγχει τον χρήστη που καλεί την μέθοδο, αν είναι Γιατρός. Σε περίπτωση που είναι Γιατρός, επιστρέφει το ID του χρήστη
- **validatePatientAndGetId(Context context);** Ελέγχει τον χρήστη που καλεί την μέθοδο, αν είναι Ασθενής. Σε περίπτωση που είναι Ασθενής, επιστρέφει το ID του χρήστη
- **handleUnauthorized(Context context, UnauthorizedException e);** Παίρνει πάνω του τον χειρισμό της μη εξουσιοδοτημένης πρόσβασης και ορίζει τα κατάλληλα HTTPS response
- **getUserTypeFromSession(Context context);** Επιστρέφει τον τύπο του χρήστη από το session

Οι μέθοδοι της κλάσης AuthUtils καλούνται μόνο από τα controllers. Τα controllers καλούν την AuthUtils με σκοπό ελέγχου πρόσβασης κάποιου.

Παράδειγμα:

Στον “PatientsController.java” υπάρχει Get request η οποία καλεί την μέθοδο “getPatients(Context context)”. Αυτό το request μπορεί να το κληθεί μόνο από Γιατρούς. Έτσι θα γίνει πρώτα κλήση του “AuthUtils.validateDoctorAndGetId(Context context);” και μόνο μετά την επιτυχής εκτέλεσης, θα γίνει κλήση του κατάλληλου service, με σκοπό να σταλθεί η λίστα από Ασθενής στον Γιατρό. Σε περίπτωση σφάλματος θα γίνει κλήση της μεθόδου “handleUnauthorized(Context context, UnauthorizedException e);”

Με αυτόν το τρόπο. Η χρήση κατάλληλων controllers με την συνδυασμού του AuthUtils. Επιτυγχάνεται ο έλεγχος παραβίασης στο Backend

Frontend:

Όπως και στην αυθεντικοποίηση, για την επικοινωνία των HTTPS requests χρησιμοποιείται “Axios”. Η αρχικοποίηση όλων των requests σε συναρτήσεις είναι στα αρχεία “src/api/patient.ts”, “src/api/doctor.ts” και “src/api/index.ts”. Αυτά τα requests χρησιμοποιούνται σε όλη την εφαρμογή.

Σημείωση: Υπάρχει επίσης άλλο ένα HTTPS request στο αρχείο “src/hooks/summary.ts”. Το request είναι Post generateSummary. Αυτό χρησιμοποιείται για την παραγωγή σχολίων με μέσω AI. Για αυτή την λειτουργία, στο backend υπάρχει και ξεχωριστό controller.

Τα HTTPS requests που προσφέρονται είναι:

- Patient
 - **Get**
 - **doctors:** Επιστροφή λίστας των γιατρών
 - **doctor:** Επιστροφή πληροφοριών του Γιατρού
 - **specialities:** Επιστροφή ιδιότητας του Γιατρού
 - **doctorRatings:** Επιστροφή σχολείων του Γιατρού
 - **appointments:** Επιστροφή λίστα από ραντεβού που έχει κάνει ο Ασθενής
 - **appointment:** Επιστροφή πληροφοριών για το ραντεβού του Ασθενή
 - **getRating:** Επιστροφή των σχολίων που άφησε ο ασθενής στον Γιατρό
 - **doctorAvailability:** Επιστροφή λίστας από ελεύθερες μέρες-ώρες του Γιατρού για ραντεβού
 - **getDiagnoses:** Επιστροφή των διαγνώσεων που έχει ο Ασθενής
 - **Post**
 - **setAppointment:** Δέσμευση ραντεβού στην επιλεγμένη μέρα-ώρα του Γιατρού
 - **setRating:** Καταχώριση σχολίου στον Γιατρό
 - **Patch**
 - **cancelAppointment:** Ακύρωση ραντεβού
 - **Put**
 - **updateRating:** Τροποποίηση του σχολίου
- Doctor
 - **Get**
 - **getAvailabilities:** Επιστροφή όλες της δυνατές ελεύθερες μέρες-ώρες που έχει ορίσει ο Γιατρός
 - **appointments:** Λίστα των ραντεβού που δεν έχουν ακόμα πραγματοποιηθεί και περιμένουν την σειρά τους
 - **appointment:** Επιστροφή πληροφοριών για το ραντεβού
 - **getPatient:** Επιστροφή πληροφοριών του Ασθενή
 - **getRatings:** Επιστροφή λίστας από σχόλια του Γιατρό
 - **Post**
 - **setDoctorAvailability:** Καταχώριση μέρα-ώρα που είναι ελεύθερος για ραντεβού ο Γιατρός
 - **setDiagnosis:** Καταχώριση διάγνωσης προς τον Ασθενή
 - **Patch**
 - **completeAppointment:** Ολοκλήρωση του ραντεβού
 - **Delete**
 - **deleteAvailabilities:** Διαγραφή μέρα-ώρα που καταχωρήθηκε ως ελεύθερη για ραντεβού

Υπάρχουν request στον user που δεν αναφέρθηκαν στο υποκεφάλαιο αυθεντικοποίηση λόγω του ότι δεν συσχετιζόντουσαν με την αυθεντικοποίηση.

- User
 - **Get**
 - **getAvatar:** Επιστροφή εικονιδίου χρήστη
 - **Post**
 - **updateAvatar:** Ανανέωση εικονιδίου χρήστη
 - **updateUserData:** Ανανέωση δεδομένων χρήστη

Κάποια στιγμιότυπα λειτουργικότητας ως παραδείγματα:

- HTTPS request appointment που έχει πρόσβαση

localhost:9191/doctor-appointments/6

Appointment Details

View and manage appointment information

Appointment A25-6

Patient



Klara Dall

Date

06/27/2025

Time

09:00 AM

Reason

I am losing hair

Status

✓ COMPLETED

Patient Information



Klara Dall

impatient@mail.com

Contact Information

☐ 6979781033

✉ impatient@mail.com

🏠 AMKA: 23451254

Back to Appointments

Diagnoses

View existing diagnoses and add new ones

Existing Diagnoses (4)

Diagnosis #1

Too much hair

Details

- Bad request appointment που δεν υπάρχει πρόσβαση (Αδυναμία επιστροφή άλλου ραντεβο)

localhost:9191/doctor-appointments/1

🏠 / doctor-appointments / 1

Appointment Details

View and manage appointment information

Appointment A25-1

Patient



Klara Dall

Date

06/29/2025

Time

05:00 PM

Reason

my teeth are in a hurt

Status

✗ CANCELLED

Patient Information



Klara Dall

impatient@mail.com

Contact Information

☐ 6979781033

✉ impatient@mail.com

🏠 AMKA: 23451254

Back to Appointments

This appointment was cancelled and cannot be modified.

- Bad request appointment που δεν υπάρχει στην βάση

Failed to fetch appointment: Appointment with ID 32 not found.

localhost:9191/doctor-appointments/32

Σημείωση: Υπάρχουν Diagrams στον φάκελο “docs/Diagrams.pdf” στο project securityBackend

4. Input Filtering

Σύμφωνα και με το OWASP Cheat Sheet που μας δόθηκε γνωρίζουμε ότι μια εφαρμογή πρέπει να διαθέτει και στο frontend και στο backend σωστό έλεγχο input για ασφάλεια του συστήματος.

Frontend

Αρχικά στο frontend έπρεπε να υλοποιήσουμε έλεγχο για όλα τα input fields με βάση την χρήση τους.

Για όλες τις συναρτήσεις ελέγχου και trimming έχουμε τα αρχεία validation.ts και sanitization.ts στον πίνακα utils.

Email

Σύμφωνα με την OWASP ο έλεγχος ενός email δεν είναι τόσο εύκολη λόγω όλων των πιθανών valid email. Για αυτό ο πιο διαδεδομένος τρόπος ελέγχου είναι το να γίνεται ένα basic email αρχικό έλεγχο και μετά να στέλνεται στον πάροχο του email για να ελέγχεται αν είναι valid.

Έτσι έχουμε την συνάρτηση validateEmail που ελέγχει ένα email:


```

export const validateEmail = (email: string): ValidationResult => {
  if (!email || email.trim().length === 0) {
    return { isValid: false, error: 'Email is required' };
  }

  const trimmedEmail = email.trim();

  // Total Length <= 254
  if (trimmedEmail.length > 254) {
    return { isValid: false, error: 'Email is too long' };
  }

  // One @ symbol
  const atCount = (trimmedEmail.match(/@/g) || []).length;
  if (atCount !== 1) {
    return { isValid: false, error: 'Email must contain exactly one @ symbol' };
  }

  const [localPart, domain] = trimmedEmail.split('@');

  if (!localPart || !domain) {
    return { isValid: false, error: 'Please enter a valid email address' };
  }

  // Local part length <= 63
  if (localPart.length > 63) {
    return { isValid: false, error: 'Email local part is too long (max 63 characters)' };
  }

  if (localPart.length === 0) {
    return { isValid: false, error: 'Email local part cannot be empty' };
  }

  // Dangerous characters: backticks, single quotes, double quotes, null bytes
  if (/['"\\u0000]/.test(trimmedEmail)) {
    return { isValid: false, error: 'Email contains invalid characters' };
  }

  // Domain part: only letters, numbers, hyphens (-), and periods (.)
  // Must have at least one period
  const domainRegex = /^[a-zA-Z0-9-]+\.[a-zA-Z]{2,}$/;
  if (!domainRegex.test(domain)) {
    return { isValid: false, error: 'Please enter a valid domain' };
  }

  return { isValid: true };
};

```

Αυτός ο βασικός έλεγχος έχει φτιαχτεί με βάση το OWASP άρθρο που μας δόθηκε. Επειδή αυτή την στιγμή στην εφαρμογή μας δεν χρησιμοποιείτε το email με κάποιο τρόπο εκτός την αυθεντικοποίηση του χρήστη, δεν έχει υλοποιηθεί το validation από τον πάροχο του email.

Password

Ο κωδικός ενός χρήστη μπορεί να έχει γράμματα, νούμερα και σύμβολα. Εδώ δεν μπορεί να γίνει κάποιος ιδιαίτερος έλεγχος όμως υλοποιήσαμε έλεγχο έτσι ώστε οι χρήστες να φτιάχνουν ισχυρούς κωδικούς (validatePassword).

FullName

Το όνομα ενός χρήστη μπορεί να χρησιμοποιεί τόνους και διάφορους χαρακτήρες αναλόγως την καταγωγή του. Για αυτό το λόγο φτιάξαμε τον παρακάτω έλεγχο:

```
export const validateFullName = (name: string): ValidationResult => {
  if (!name || name.trim().length === 0) {
    return { isValid: false, error: 'Full name is required' };
  }

  const trimmedName = name.trim();

  if (trimmedName.length < 2) {
    return { isValid: false, error: 'Full name must be at least 2 characters' };
  }

  if (trimmedName.length > 100) {
    return { isValid: false, error: 'Full name is too long' };
  }

  const nameRegex = /^[a-zA-Zñáéíóúüàèìòùâêîôûäëïöÿñ\s\-' ]+$/;
  if (!nameRegex.test(trimmedName)) {
    return { isValid: false, error: 'Full name contains invalid characters' };
  }

  return { isValid: true };
};
```

PhoneNumber

Ένα τηλέφωνο μπορεί να έχει ελληνικό ή international form καθώς μπορεί ο user να έχει τηλέφωνο από άλλη χώρα. Επιπλέον το τηλέφωνο δεν είναι υποχρεωτικό πεδίο οπότε μπορεί να είναι και κενό. Για αυτό τον λόγο έχουμε τον παρακάτω έλεγχο:

```
export const validatePhone = (phone: string): ValidationResult => {
  if (!phone || phone.trim().length === 0) {
    return { isValid: true };
  }

  const trimmedPhone = phone.trim();

  const phoneRegex = /^[+]?[0-9]{7,15}$/;
  if (!phoneRegex.test(trimmedPhone.replace(/[ \s\-\(\)]/g, ''))) {
    return { isValid: false, error: 'Please enter a valid phone number' };
  }

  return { isValid: true };
};
```

AMKA

Ο ΑΜΚΑ αποτελείται από 11 ψηφία. Έτσι ο έλεγχος του είναι αρκετά απλός:

```
export const validateAMKA = (amka: string): ValidationResult => {
  if (!amka || amka.trim().length === 0) {
    return { isValid: false, error: 'AMKA is required' };
  }

  const trimmedAMKA = amka.trim();

  if (!/^d{11}$/.test(trimmedAMKA)) {
    return { isValid: false, error: 'AMKA must be 11 digits' };
  }

  return { isValid: true };
};
```

LicenceID

Στο licence id γίνεται ένα απλό τσεκ για το αν είναι valid.

```
export const validateLicenseID = (licenseID: string): ValidationResult => {
  if (!licenseID || licenseID.trim().length === 0) {
    return { isValid: false, error: 'License ID is required' };
  }

  const trimmedLicense = licenseID.trim();

  if (trimmedLicense.length < 3) {
    return { isValid: false, error: 'License ID is too short' };
  }

  if (trimmedLicense.length > 50) {
    return { isValid: false, error: 'License ID is too long' };
  }

  const licenseRegex = /^[a-zA-Z0-9\-\_\/]+$/;
  if (!licenseRegex.test(trimmedLicense)) {
    return { isValid: false, error: 'License ID contains invalid characters' };
  }

  return { isValid: true };
};
```

FreeForm

Υπάρχουν διάφορα πεδία στην εφαρμογή μας που μπορούν να θεωρηθούν free form text (text που μπορεί να χρειαστεί διάφορους χαρακτήρες χωρίς να θεωρείται παράξενο). Για αυτό το λόγο εδώ το validation είναι πιο δύσκολο.

Για αυτό δημιουργήσαμε μια πιο γενική συνάρτηση την `validateFreeFormText` που τις δίνουμε ειδικές οδηγίες αναλόγως τι πεδίο ελέγχει. Το κείμενο πάντα περνάει από normalization όπως μας ζητείται στο OWASP για έλεγχο για invalid characters.

```
export const validateFreeFormText = (
  text: string,
  minLength: number = 0,
  maxLength: number = 1000,
  required: boolean = false,
  fieldName: string = 'Text',
): ValidationResult => {
  const allowedCategories = new Set(['L', 'N', 'Z', 'M']);
  const allowedIndividualChars = new Set([
    '"', '-', '.', ',', '!', '?', ':', '(', ')', '"', '/', '&', '_', ' ', '\n'
  ]);
  const disallowedChars = new Set(['\u0000']);

  if (required && (!text || text.trim().length === 0)) {
    return { isValid: false, error: `${fieldName} is required` };
  }

  if (!text || text.trim().length === 0) {
    return { isValid: true };
  }

  const normalizedText = normalizeText(text);

  if (minLength > 0 && normalizedText.trim().length < minLength) {
    return {
      isValid: false,
      error: `${fieldName} must be at least ${minLength} characters`,
    };
  }

  if (maxLength > 0 && normalizedText.length > maxLength) {
    return {
      isValid: false,
      error: `${fieldName} must not exceed ${maxLength} characters`,
    };
  }

  const invalidChar = detectInvalidCharacter(
    normalizedText,
    allowedCategories,
    allowedIndividualChars,
    disallowedChars,
  );

  if (invalidChar) {
    return {
      isValid: false,
      error: `${fieldName} contains invalid character '${invalidChar}'`,
    };
  }

  return { isValid: true };
};
```

Με αυτά και το απλό sanitization που έχουμε κάνει implement έχουμε καταφέρει έλεγχο input στο frontend . Όμως το πιο σημαντικό input verification είναι στο backend καθώς υπάρχουν τρόποι και εργαλεία κάποιος κακόβουλα να παρακάμψει το UI της εφαρμογής και να κάνει επίθεση στο API της.

Backend

Το backend της εφαρμογής είναι φτιαγμένο με java.

Για την ασφάλεια του αρχικά δημιουργίσαμε 3 νέα αρχεία στον φάκελο utils.

Το αρχείο **InputValidator.java** έχει όλα τα regex και τις συναρτήσεις που θα χρησιμοποιήσουμε για να ελέγξουμε ότι το input δεν είναι κακόβουλο.

Regex Patterns:

```
private static final Pattern EMAIL_PATTERN = Pattern.compile(
    regex: "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}$"
);

private static final Pattern PHONE_PATTERN = Pattern.compile(
    regex: "^[\\d\\s+()\\-]{7,20}$"
);

private static final Pattern AMKA_PATTERN = Pattern.compile(
    regex: "^\\d{11}$"
);

private static final Pattern LICENSE_PATTERN = Pattern.compile(
    regex: "^[A-Z0-9]{1,20}$"
);

private static final Pattern NAME_PATTERN = Pattern.compile(
    regex: "^[a-zA-Z\\s'-]{2,100}$"
);

private static final Pattern PASSWORD_PATTERN = Pattern.compile(
    regex: "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,}$"
);

private static final Pattern SQL_INJECTION_PATTERN = Pattern.compile(
    regex: "('|(\\-\\-)|(;)|(\\|\\|)|(|\\*)|(\\*\\*)|(xp_|(sp_))",
    Pattern.CASE_INSENSITIVE
);

private static final Pattern HTML_PATTERN = Pattern.compile(
    regex: "<[^>]*>|&[a-zA-Z]*;|javascript:|on\\w+\\s*=",
    Pattern.CASE_INSENSITIVE
);
```

Κάτω από τα patterns ακολουθούν απλές συναρτήσεις που ελέγχουν κατά πόσο το input ακολουθεί το σωστό regex.

Στο **InputFilter.java** κάνουμε το sanitization των inputs ώστε να αφαιρέσουμε κακόβουλους χαρακτήρες ή στοιχεία κακόβουλου κώδικα.

```

public static String sanitizeString(String input) {
    if (input == null) {
        return "";
    }

    return input
        .trim()
        .replaceAll(regex: "<[>]*>", replacement: "")// HTML tags
        .replaceAll(regex: "javascript:", replacement: "")// javascript
        .replaceAll(regex: "on\\w+\\s*=", replacement: "")// event handlers
        .replaceAll(regex: "&[a-zA-Z]+;", replacement: "")// HTML entities
        .replaceAll(regex: "[\\x00-\\x08\\x0B\\x0C\\x0E-\\x1F]", replacement: ""); // control characters
}

```

Επιπλέον εδώ γίνεται και validation για το avatar του χρήστη .

```

public static String sanitizeAvatar(String avatar) {
    if (avatar == null) {
        return null;
    }
    String sanitized = avatar.trim();

    // Validate it's either a valid URL or base64
    if (sanitized.startsWith(prefix: "http://") || sanitized.startsWith(prefix: "https://")) {
        // no javascript: allowed
        if (sanitized.contains(s: "javascript:")) {
            throw new ValidationException(message: "Invalid avatar URL");
        }
        return sanitized;
    } else if (isValidBase64(sanitized)) {
        return sanitized;
    } else {
        throw new ValidationException(message: "Avatar must be a valid URL or base64 encoded image");
    }
}

```

Οι έλεγχοι ακολουθούν τους ελέγχους του frontend καθώς και έχουν και πιο ανεπτυγμένους ελέγχους.

Το **ValidationException.java** απλά ενημερώνει τον χρήστη για προβλήματα στο validation των στοιχείων του.

Η εφαρμογή πλέον έχει αναβαθμιστεί με βάση όλους αυτούς τους ελέγχους input. Ο κώδικας στις φωτογραφίες από πάνω είναι ενδεικτικός. Μπορείτε να δείτε τον υπόλοιπο στο φάκελο του app.

5. Εύρεση ευπαθειών ασφάλειας (ZAP)

Για να βρεθούν ευπάθειες χρησιμοποιήθηκε το εργαλείο zap και συγκεκριμένα οι λειτουργίες

- **passive scan:** ελέγχει τα http request όσο γίνεται χρήση της ιστοσελίδας
- **ajax spider:** (OWASP ZAP) ελέγχει όλα τα links που μπορεί να βρεί ακόμα και αυτά που θα είναι από τον client

- **active mode:** χρησιμοποιείτε για εύρεση vulnerabilities με γνωστές επιθέσεις

Ανοίγοντας την ιστοσελίδα μέσω του manual explore που προσφέρει το zap στα alerts βλέπουμε αυτά τα risks

```
> 🚩 Content Security Policy (CSP) Header Not Set
> 🚩 Missing Anti-clickjacking Header
> 🚩 Application Error Disclosure
> 🚩 Server Leaks Version Information via "Server" HTTP Response Header Field (Systemic)
> 🚩 Strict-Transport-Security Header Not Set (Systemic)
> 🚩 Timestamp Disclosure - Unix
> 🚩 X-Content-Type-Options Header Missing (Systemic)
```

Content Security Policy (CSP) Header Not Set – Risk Medium

Περιγραφή: Χρησιμοποιείτε για Integrity. Ο client έχοντας την πολιτική μπορεί να ξέρει τι να δέχεται και τι όχι. Έτσι βοηθά στην αποτροπή επιθέσεων Cross-Site Scripting (XSS), clickjacking και άλλων επιθέσεων εισαγωγής κώδικα.

Επίλυση: Στο config file του nginx έχει προστεθεί header content security policy με αυτές τις προδιαγραφές.

```
# Set Content Security Policy directives (see MDN docs)
add_header Content-Security-Policy "
    default-src 'self';
    child-src 'none';
    connect-src 'self';
    font-src 'self';
    frame-src 'none';
    img-src 'self' data;;
    manifest-src 'self';
    media-src 'none';
    object-src 'none';
    script-src 'self' 'nonce-$cspNonce';
    script-src-elem 'self' 'nonce-$cspNonce';
    script-src-attr 'self' 'nonce-$cspNonce';
    style-src 'self' 'nonce-$cspNonce';
    style-src-elem 'self' 'nonce-$cspNonce';
    style-src-attr 'self' 'unsafe-inline';
    base-uri 'self';
    worker-src 'self';
    form-action 'self';
    frame-ancestors 'none';
" always;
```

```
# Set a unique nonce for CSP using the request ID
set $cspNonce $request_id;

# Replace all occurrences of NGINX_NONCE in HTML with the generated nonce
sub_filter_once off;
sub_filter_types text/html;
sub_filter 'NGINX_NONCE' $cspNonce;
```

Επειδή στην εφαρμογή γίνεται χρήση inline. Χρειάστηκε να προστεθεί nonce για υπάρχει ορισμός ποια inline μπορούν να χρησιμοποιηθούν και ποια όχι.

Server Leaks Version Information via “Server” HTTP response Header Field (Systemic) - Risk Low

Περιγραφή: Υπάρχει παρουσίαση της έκδοσης του nginx στον Header. Αυτό μπορεί να χρησιμοποιηθεί μελλοντικά για εύρεση vulnerabilities πάνω στην έκδοση που χρησιμοποιείτε.

Επίλυση: Στον config file του nginx προστέθηκε ρύθμιση που απενεργοποιεί την παρουσίαση έκδοσης στον header.

```
# Remove the server version from the response header
server_tokens off;
```

Strict-Transport-Security Header – Risk Low

Περιγραφή: Ο χρήστης μπορεί να γίνει σύνδεση μέσου πρωτοκόλλου HTTP αντί για HTTPS. Παρόλου που μπορεί να γίνεται αναδιαδρομή στην ιστοσελίδα που ακολουθεί το πρωτόκολλο HTTPS. Θα πρέπει να γνωρίζει ότι η πρόσβαση γίνεται μόνο μέσω HTTPS και όχι HTTP. Επιπλέον, σε μελλοντικές συνδέσεις με τον κεντρικό υπολογιστή, το πρόγραμμα περιήγησης δεν θα επιτρέπει στον χρήστη να παρακάμπτει σφάλματα ασφαλούς σύνδεσης, όπως ένα μη έγκυρο πιστοποιητικό

Επίλυση: Στο config file του nginx έχει προστεθεί header strict transport security

```
# Enable HSTS for security (force HTTPS in browsers)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
```

(Χρήση του HSTS για έναν χρόνο - Ισχύουν οι κανόνες για όλα τα subdomains)

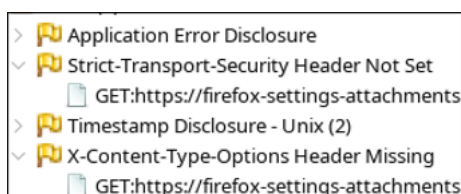
X-Content-Type-Options Header Missing (Systemic) - Risk Low

Περιγραφή: Για λόγους όπως παλιός browser. Μπορεί να γίνει προσπάθει να μαντέψει τον τύπο. Αυτό θα μπορούσε να μεταφέρει στον client κακόβουλο πρόγραμμα το οποίο θα μπορεί να εκτελεστεί. Για αυτό θα πρέπει να υπάρχει αυστηρή λειτουργία με τους τύπους

Επίλυση: Στο config file του nginx έχει προστεθεί header X content type options

```
# Prevent browsers from MIME-sniffing the response
add_header X-Content-Type-Options "nosniff" always;
```

Με αυτούς του τρόπους αυξήσαμε την ασφάλεια και το zap δεν μας τα εμφανίζει.



Τώρα για να μπορέσουμε να τρέξουμε το AJAX Spider και το Active Scan θα χρειαστεί να δώσουμε την δυνατότητα στο ZAP να μπορεί να συνδεθεί στην εφαρμογή. Πρώτα σαν Patient και μετά σαν Doctor

Τα βήματα για την ρύθμιση του ZAP είναι

- Προσθέτουμε στο Default Context το url της εφαρμογή μας
- Φτιάχνουμε καινούργιο Context για να κάνουμε scan με τις προδιαγραφές που θα βάλουμε
- Προσθέτουμε στο καινούργιο Context το url της εφαρμογή μας
- Θα χρειαστεί να γίνει σύνδεση να να εμφανιστεί το POST που στάλθηκε, ώστε να προστεθεί στο καινούργιο Context. Αυτό χειρίζεστε ώστε το ZAP να ξέρει πως να στείλει το POST request.
 - ο Στην εφαρμογή αυτό γίνεται μέσα από JSON-base Auth Login Request

The screenshot shows the '5: Authentication' panel in ZAP. It contains the following fields and settings:

- Currently selected Authentication method for the Context:** JSON-based Authentication
- Configure Authentication Method**
 - Login Form Target URL *:** https://localhost:9191/api/login
 - URL to GET Login Page:** https://localhost:9191/api/login
 - Login Request POST Data *:** {"email": "test232@test.com", "password": "qwerQWER1234!@!@"}
 - Username Parameter *:** email
 - Password Parameter *:** password

- Έμεινε να προσθέσουμε τον User στο καινούργιο Context πριν αρχίσουμε τους ελέγχους

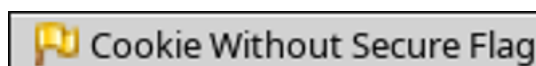
The screenshot shows the '5: Users' panel in ZAP. It contains a table with the following data:

Enabled	ID	Name ^
☒	1,...	test232@test.com
☒	1,...	test667@test.com

Buttons: Add..., Modify..., Remove

Τώρα μπορούμε να αρχίσουμε τους ελέγχους. Πατώντας πάνω στο καινούργιο context επιλέγουμε πρώτα AJAX Spider και μετά Active Scan.

Εμφανίστηκε καινούριο alert με risk – low.



Cookie Without Secure Flag – Risk Low

Περιγραφή: Υπάρχει μεταφορά των cookies μέσα από μη ασφαλές κανάλι κάτι το οποίο θα μπορούσε να είναι λόγος leak ευαίσθητων δεδομένων που μεταφέρουν τα cookies. πχ το session του χρήστη.

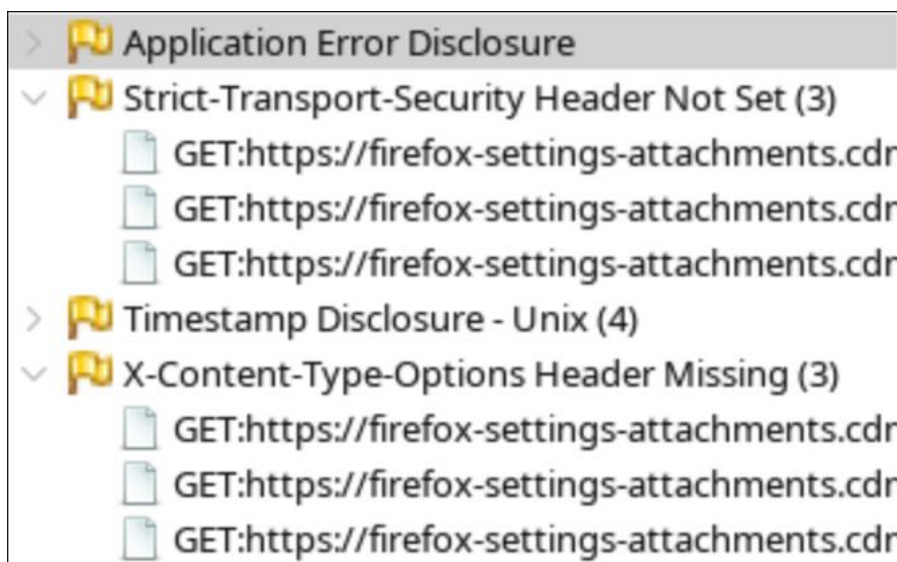
Επίλυση: Στο config file του nginx έχουν προστεθεί οι εξής ρυθμίσεις:

```
# Indicate that the request is coming from the Nginx proxy
proxy_set_header X-NginX-Proxy true;
# Indicate that HTTPS is used for the original request
proxy_set_header X-Forwarded-Proto https;
```

Ελέγχει τα requests από που προέρχονται.

Ορισμός καναλιού για cookies που στέλνεται από το backend.

Με αυτούς τους τρόπους αν γίνουν έλεγχοι θα φαίνεται ότι δεν υπάρχει προβλήματα που χρειάζονται απασχόληση.



Εξαιρέσεις:

- **Timestamp Disclosure – Unix:** Παρόλου που επιστρέφει Low Risk. Στην περίπτωση μας δεν είναι θέμα διότι δεν είναι ευαίσθητη πληροφορία
- **Application Error Disclosure:** Παρόλου που επιστρέφει Low Risk. Το error δεν επιστρέφει κάποια πληροφορία που θα μπορούσε να χρησιμοποιηθεί.

6. Τελική Μελέτη ασφάλειας ΠΣ

Vulnerability post assessment

Κατά την διάρκεια λύσης vulnerabilites της εφαρμογής. Έγιναν διάφορες αναβαθμίσεις στα npm packages της εφαρμογής (frontend), καθώς και αναβάθμιση των λειτουργικών συστημάτων. Για αυτό τον λόγο, πολλά από τα προβλήματα που είχαν βρεθεί έχουν λυθεί.

Τα vulnerabilities που είχαν βρεθεί είναι τα εξής:

Λογισμικό	Αδυναμία	CVSS 3.1 Score
Debian 12 sudo 1.9.13p3	CVE-2025-32463	7.8 (High)
Docker Compose 2.40.0	CVE-2025-62725	8.9 (High)
JDK 21	CVE-2024-20952	7.4 (High)
Eclipse Jetty 11.0.5	CVE-2025-5115	7.7 (High)
Eclipse Jetty 11.0.5	CVE-2021-34429	5.3 (Medium)

Έγινε αναβάθμιση από Docker Compose 2.40.0 σε 2.40.3

Μέσα στα DockerFile έχουν οριστεί οι καινούργιες εκδόσεις των images. Οι αλλαγές

- Debian 13 (bookworm) στο NGINX
- Ubuntu 24.04.3 LTS στο eclipse-temurin
- OpenJDK 25.0.2 2026-01-20 TLC στο eclipse-temurin

Για το Eclipse Jetty 11.0.5. έγινε αναβάθμιση της βιβλιοθήκης Javalin σε javalin 6.6.0, και πλέον χρησιμοποιείται η έκδοση Eclipse Jetty 12

Λόγω των παραπάνω αναβαθμίσεων, δεν έχουν εντοπιστεί νέες vulnerabilities

Risk post assessment

Έχουμε τον παρακάτω πίνακα αποτίμησης κινδύνου:

Απειλή	Επίπεδο Συνέπειας	Αποτίμηση Απειλής	Αποτίμηση επιπέδου κινδύνου
Unauthorized Access	High (Πληροφορίες χρηστών, ιατρικό απόρρητο)	0	0
Ransomware	High (Καταστροφή δεδομένων, μη λειτουργικότητα συστήματος)	0	0
Web Defacement	Medium (Δυσφήμιση εταιρίας)	0	0
Code Injection	High (Καταστροφή δεδομένων, unauthorized access)	0	0
Denial of Service	High (Μη λειτουργικότητα του συστήματος)	0	0

Σχολεία:

Μετά την υλοποίηση των μέτρων ασφάλειας και τις αναβαθμίσεις λογισμικού, δεν έχουν εντοπιστεί πλέον ενεργές vulnerabilities. Ως αποτέλεσμα, το επίπεδο υπολειπόμενου κινδύνου για τις παραπάνω απειλές είναι μηδενικές.

Το σύστημα θεωρείται προς το παρόν ασφαλές, ωστόσο απαιτείται συνεχής παρακολούθηση και τακτικός έλεγχος για την αντιμετώπιση μελλοντικών απειλών